

Question 1: Heuristic design and admissibility

Steps heuristic:

pseudocode:

```
for position in currState
    if currState[position] != goalState[position]
        count++
```

The unavoidable remaining cost for this heuristic is the number of positions where the letters in the goal and current state don't match. Only a single letter can change per action, so the minimum remaining cost is the minimum number of moves to get the goal state. The optimal route guarantees at least the number of steps, as there are different positions where the letters are different between the goal and current state since only one letter can change per action. The heuristic value increases the cost by one for each different position between the two states. The heuristic value is the minimum that the optimal route can be, but the optimal route may need to take more actions to reach the goal state, so the heuristic cannot overestimate the true optimal remaining cost.

Scrabble heuristic:

pseudocode:

```
for position in currState
    if currState[position] != goalState[position]
        count += scrabbleValgoalState[position]
```

The unavoidable remaining cost for this heuristic is the sum of all Scrabble values for the letters that need to be replaced in the current state to match the goal state. Only one letter can be updated at a time, so in order to reach the goal state, they individually have to replace a letter at some point, and each of these actions comes with its Scrabble value cost. The optimal route has to at least change the letters that are mismatched at some point to reach this goal state. This yields a minimum total cost equal to the sum of the new letters that replaced the old ones. This minimum-cost optimal route may be equivalent to or larger than the heuristic function, but never less. Therefore, the heuristic cannot overestimate the true optimal remaining cost.

Frequency heuristic:

pseudocode:

```
for position in currState
    if currState[position] != goalState[position]
        cost++
if cost > 0
    cost += goalFrequencyCost
```

The unavoidable remaining cost for this heuristic is the frequency cost of the goal word plus the total number of mismatched positions between the current and goal state. The optimal route, on the other hand, totals the sum of actions taken, plus the sum of the frequency costs of all of the states that it took to reach the goal (including the goal). This optimal route will always cost at least as much as the heuristic function because it includes all the frequency costs of the states taken to get to the goal state, so the heuristic cannot overestimate the true optimal remaining cost.

Question 2: Does the heuristic actually help?

Word Pair	Cost Measure	UCS Cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction (%)
WHY→ASK	Steps	5	773	5	16	97.93
WHY→ASK	Scrabble	9	765	9	13	98.30
WHY→ASK	Frequency	38.8554	808	38.8554	634	21.53
BEAR→DUCK	Steps	4	691	4	6	99.13
BEAR→DUCK	Scrabble	11	2981	11	6	99.80
BEAR→DUCK	Frequency	38.2402	1084	38.2402	302	72.14

1. Yes, UCS and A* returned the same optimal cost in every case.
2. For WHY→ASK, Scrabble had the largest percentage reduction with 98.3%. For BEAR→DUCK, Scrabble once again had the largest percentage reduction with 99.8%.

3. For WHY→ASK, Frequency was the cost measure with the least informative heuristic because it had the smallest reduction percentage of 21.53%. For BEAR→DUCK, Frequency was once again the cost measure with the least informative heuristic because it had the smallest reduction percentage of 72.14%.
4. My Frequency heuristic is the least informative because it is the least similar to the actual cost to reach the goal state from the current state compared to the other heuristics. My Step heuristic is very similar to the actual Step cost since it counts every letter that needs to change, while the actual cost is simply the number of actions taken, and the letters that need to change will cost an action at some point. My Scrabble heuristic is very similar in the sense that it takes the letters that need to change and their Scrabble costs into account, which is what contributes to the actual Scrabble cost. My Frequency heuristic ignores the Frequency cost of all the states required to be visited to reach the goal, leading it to be the least informative.

Question 3: Reversibility and direction-dependent costs

Steps:

Yes, the optimal cost from $W1$ to $W2$ must always equal the optimal cost from $W2$ to $W1$. Every action has the same price of 1, so every single step reversed has the same cost forward and backwards.

Scrabble:

No, the cost of some moves when reversed is not always the same. An example of this is going from *sap* to *map*: the S gets replaced with an M, which has a Scrabble cost of 3, so this action has a cost of 3. When reversing the action to do *map* to *sap*, the M gets replaced with an S, which has a Scrabble cost of 1, so this action only has a cost of 1. Since the two actions do not have the same costs, the optimal cost from $W1$ to $W2$ is not always equal to the optimal cost from $W2$ to $W1$ here.

Frequency:

No, the cost of some moves when reversed is not always the same. An example from the provided dictionary is with *hit* and *sit*. Going from *hit* to *sit*, the cost increase will be $1 + 5.023744 = 6.023744$. Going from *sit* to *hit*, on the other hand, the cost increase will be $1 + 3.3941658 = 4.3941658$. Since these costs are different when reversing the action, the optimal cost from $W1$ to $W2$ is not always equal to the optimal cost from $W2$ to $W1$ here.

Question 4: LLM heuristic audit

1. I used Claude Sonnet 5 Medium, and here is the prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Here is the proposed heuristic:

The proposed heuristic

$$h_{\text{rar}}(n) = \begin{cases} 0 & n = \text{goal} \\ (h_0(n) - 1)(1 + r_{\min}(L)) + (1 + \text{rarity}(\text{goal})) & n \neq \text{goal} \end{cases}$$

- 2.

pseudocode:

k = number of mismatches between currState and goalState

if k == 0:

return 0

rMin = minimum rarity of any dictionary word with the same length as currState

return (k - 1)(1 + rMin) + (1 + rarity(goal))

3. The heuristic is admissible. If there are k mismatches between the goal and the current state, then k is the minimum number of moves required to reach the goal state since only one letter can be changed per action. The $(1 + \text{rarity}(\text{goal}))$ makes sense since the frequency cost of the goal state needs to be added when the goal is reached, so it's inevitable. The $(1 + r_{\min})$ term comes from taking the minimum frequency cost possible with the same string length. This cost can be taken a minimum of $(k - 1)$ times, so the two values get multiplied. This is the lower bound on the minimum possible cost to get from the current state to the goal state. The actual cost will at least need to be as much as the heuristic, so the heuristic never overestimates the actual cost.

4. The heuristic is admissible, so no repair is needed.
5. The proposed heuristic resulted in the same solution cost with fewer nodes expanded. For WHY→ASK, both heuristics had a total cost of 38.8554, but the total number of nodes expanded went from 755 to 634 using the proposed heuristic. For BEAR→DUCK, both heuristics had a total cost of 38.2402, but the total number of nodes expanded went from 720 to 257 using the proposed heuristic. This shows that the proposed heuristic is more informative than the original, since it resulted in fewer nodes being expanded while keeping the cost.

AI disclosure: I used ChatGPT to help clarify parts of the assignment and review my work. I verified the results using the provided test files and required experiments.